

School of Science and Engineering

AI624 AI on Edge Devices

ASSIGNMENT 1

Due Date: 11:59 PM, 13th September 2026.

Format: 4 tasks worth a total of 100 marks (Task 1: 15, Task 2: 20, Task 3: 25, Task 4: 40)

Instructions:

- Collaboration with peers is allowed, but copying another student's solution is strictly prohibited. This is not a group assignment—each student must submit their own work.
 - Submit all work in a single Jupyter notebook. All computations, reported values, results, plots, and analysis for every task and subtask must be clearly presented within the notebook using Markdown cells and code cells as appropriate.
 - Keep the notebook clean and well organized using clear section and subsection headings corresponding to each task and subtask. All reported quantities should be presented neatly, with explanations where required.
 - Submit your work on LMS by the deadline.
 - If you have any concerns, contact the instructor or teaching assistants. You are also encouraged to use the assignment Slack channel for questions and discussion.
- You are among the most competent individuals in the country. Do not let plagiarism undermine your learning. Any instance of plagiarism will be dealt with according to the university's rules and regulations.
 - You must acknowledge any use of generative AI tools. Include the following statement where applicable:
I have used [insert Tool Name] to [write, generate, plot, or compute; explain the specific use of generative AI] [number of times].
 - You may optionally share a link to the relevant chatbot conversation for further clarity.

Background

This section summarizes the operations and efficiency concepts required throughout the assignment.

MACs and FLOPs

A multiply-accumulate operation performs one multiplication followed by one addition. Throughout this assignment,

$$1 \text{ MAC} \approx 2 \text{ FLOPs.}$$

Therefore,

$$\text{FLOPs} \approx 2 \times \text{MACs.}$$

Bias additions may be ignored when computing MACs and FLOPs unless otherwise stated.

Convolutional Operations

Consider an input tensor

$$X \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}.$$

Assume stride $S = 1$ and padding such that the spatial dimensions are preserved.

Standard Convolution

A standard convolution uses a different $K \times K$ kernel for every input-output channel pair.

The weight tensor is

$$W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K \times K}.$$

Each output channel combines information from all input channels:

$$Y_{c_o} = \sum_{c_i=1}^{C_{\text{in}}} W_{c_o, c_i} * X_{c_i}.$$

The number of weights is

$$N_{\text{weights}}^{\text{standard}} = C_{\text{out}} C_{\text{in}} K^2.$$

Including one bias for each output channel,

$$N_{\text{params}}^{\text{standard}} = C_{\text{out}} C_{\text{in}} K^2 + C_{\text{out}}.$$

The number of MACs is

$$\text{MACs}^{\text{standard}} = B H W C_{\text{out}} C_{\text{in}} K^2.$$

Standard convolution performs both spatial filtering and channel mixing.

Depthwise Convolution

A depthwise convolution applies one independent $K \times K$ filter to each input channel:

$$Y_c = W_c * X_c.$$

The weight tensor is

$$W_{\text{DW}} \in \mathbb{R}^{C_{\text{in}} \times 1 \times K \times K}.$$

The number of weights is

$$N_{\text{weights}}^{\text{DW}} = C_{\text{in}} K^2.$$

Including biases,

$$N_{\text{params}}^{\text{DW}} = C_{\text{in}} K^2 + C_{\text{in}}.$$

The number of MACs is

$$\text{MACs}^{\text{DW}} = B H W C_{\text{in}} K^2.$$

Depthwise convolution performs spatial filtering independently for each channel but does not mix information between channels.

In PyTorch, depthwise convolution can be implemented using

$$\text{groups} = C_{\text{in}}.$$

Pointwise Convolution

A pointwise convolution is a 1×1 convolution used to mix information across channels at each spatial location.

Its weight tensor is

$$W_{\text{PW}} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 1 \times 1}.$$

The number of weights is

$$N_{\text{weights}}^{\text{PW}} = C_{\text{in}} C_{\text{out}}.$$

Including biases,

$$N_{\text{params}}^{\text{PW}} = C_{\text{in}} C_{\text{out}} + C_{\text{out}}.$$

The number of MACs is

$$\text{MACs}^{\text{PW}} = B H W C_{\text{in}} C_{\text{out}}.$$

Depthwise Separable Convolution

A depthwise separable convolution consists of

$$X \xrightarrow{\text{Depthwise } K \times K} Z \xrightarrow{\text{Pointwise } 1 \times 1} Y.$$

The depthwise stage performs spatial filtering and the pointwise stage performs channel mixing.

The number of weights is

$$N_{\text{weights}}^{\text{DS}} = C_{\text{in}}K^2 + C_{\text{in}}C_{\text{out}}.$$

Including biases from both operations,

$$N_{\text{params}}^{\text{DS}} = C_{\text{in}}K^2 + C_{\text{in}} + C_{\text{in}}C_{\text{out}} + C_{\text{out}}.$$

The total number of MACs is

$$\text{MACs}^{\text{DS}} = BHW (C_{\text{in}}K^2 + C_{\text{in}}C_{\text{out}}).$$

Attention and Transformers

Consider an input sequence

$$X \in \mathbb{R}^{N \times d},$$

where N is the sequence length and d is the embedding dimension.

For multi-head self-attention with h heads,

$$d_h = \frac{d}{h}.$$

The query, key, and value projections are

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d}.$$

Self-attention is computed as

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_h}} \right),$$

followed by

$$O = AV.$$

The concatenated attention output is projected using

$$W_O \in \mathbb{R}^{d \times d}.$$

A common Transformer MLP has dimensions

$$d \rightarrow 4d \rightarrow d.$$

The main MAC counts are

$$\text{MACs}_{QKV} = 3Nd^2,$$

$$\text{MACs}_{QK^\top} = N^2d,$$

$$\text{MACs}_{AV} = N^2d,$$

$$\text{MACs}_O = Nd^2,$$

and

$$\text{MACs}_{\text{MLP}} = 8Nd^2.$$

Projection and MLP operations scale linearly with sequence length, while the attention matrix operations contain terms that scale as

$$\mathcal{O}(N^2).$$

KV Cache

During autoregressive generation, tokens are generated one at a time.

Without a KV cache, the key and value projections for the previous t tokens are recomputed at every decoding step:

$$\text{MACs}_{K,V}^{\text{no cache}} = 2td^2.$$

With a KV cache, previously computed keys and values are stored. Only the key and value corresponding to the new token need to be computed:

$$\text{MACs}_{K,V}^{\text{cache}} = 2d^2.$$

The stored tensors are

$$K_{\text{cache}} \in \mathbb{R}^{t \times d}, \quad V_{\text{cache}} \in \mathbb{R}^{t \times d}.$$

For a single Transformer layer, batch size B , and b bytes per element,

$$M_{KV} = 2Btdb.$$

For $B = 1$,

$$M_{KV} = 2tdb.$$

KV caching reduces repeated computation during decoding at the cost of additional memory.

Model Efficiency Metrics

For matrix multiplication

$$Y = XW,$$

where

$$X \in \mathbb{R}^{B \times M}, \quad W \in \mathbb{R}^{M \times N},$$

the output is

$$Y \in \mathbb{R}^{B \times N}.$$

The number of MACs is

$$\text{MACs} = BMN,$$

and

$$\text{FLOPs} = 2BMN.$$

If each tensor element occupies b bytes and X and W are each read once while Y is written once,

$$\text{Bytes Moved} = b(BM + MN + BN).$$

Arithmetic intensity is

$$I = \frac{\text{FLOPs}}{\text{Bytes Moved}}$$

and is measured in FLOPs/byte.

For the matrix multiplication above,

$$I(B) = \frac{2BMN}{b(BM + MN + BN)}.$$

Increasing arithmetic intensity means performing more arithmetic for each byte transferred from memory.

Memory-Bound and Compute-Bound Execution

Real hardware has finite memory bandwidth and finite compute throughput.

Let

$$P_{\text{peak}}$$

denote peak compute throughput in FLOP/s and let

$$\beta$$

denote memory bandwidth in bytes/s.

An idealized upper bound on achievable throughput is

$$P = \min(P_{\text{peak}}, \beta I).$$

At low arithmetic intensity, performance is primarily limited by memory bandwidth and approximately follows

$$P \approx \beta I.$$

At sufficiently high arithmetic intensity, performance approaches the compute limit

$$P \approx P_{\text{peak}}.$$

The transition occurs approximately at

$$I^* = \frac{P_{\text{peak}}}{\beta}.$$

The measured throughput of an operation is

$$P_{\text{achieved}} = \frac{\text{FLOPs}}{t_{\text{measured}}}.$$

A plot of achieved throughput against arithmetic intensity can therefore be used to observe the transition from memory-bound to compute-bound execution.

Profiling

Analytical metrics such as parameters, MACs, FLOPs, and arithmetic intensity describe theoretical computational cost. Actual hardware performance also depends on memory movement, kernel implementation, hardware utilization, numerical precision, and operator shape.

Useful measured quantities include:

- inference latency,
- peak and average memory usage,
- CPU/GPU utilization,
- power consumption,
- energy consumption.

Energy over an inference interval may be estimated as

$$E = \int_{t_0}^{t_1} P(t) dt,$$

or using discrete power measurements,

$$E \approx \sum_i P_i \Delta t.$$

Before recording measurements, run several warm-up iterations and average results over multiple forward passes.

No dataset is required for the profiling tasks. Synthetic tensors can be generated directly using PyTorch:

```
x = torch.randn(shape, device=device)
```

Use the same device, numerical precision, and profiling procedure when comparing related experiments.

1 Task 1: Convolutional Neural Networks

Consider

$$X \in \mathbb{R}^{1 \times 64 \times 56 \times 56},$$

with

$$C_{\text{out}} = 128, \quad K = 3, \quad S = 1, \quad P = 1.$$

Assume FP32 weights and activations.

For each of the following:

1. Standard 3×3 convolution from 64 to 128 channels.
2. Depthwise 3×3 convolution on 64 channels.
3. Pointwise 1×1 convolution from 64 to 128 channels.
4. Depthwise 3×3 convolution followed by pointwise 1×1 convolution from 64 to 128 channels.

Deliverables. Compute and report:

- output tensor dimensions,
- number of weights,
- total number of parameters including biases,
- MACs,
- FLOPs,

For the depthwise separable convolution, calculate its parameter and MAC reduction relative to the standard convolution:

$$\text{Parameter Reduction} = 1 - \frac{N_{\text{params}}^{\text{DS}}}{N_{\text{params}}^{\text{standard}}},$$

$$\text{MAC Reduction} = 1 - \frac{\text{MACs}^{\text{DS}}}{\text{MACs}^{\text{standard}}}.$$

2 Task 2: Attention and Transformers

Part (a): Transformer Computation

Consider a single Transformer block with

$$N = 512, \quad d = 768, \quad h = 12.$$

Assume FP16 tensors. Ignore biases, LayerNorm, activation functions, and softmax when computing MACs.

Deliverables. Compute and report:

1. The dimension d_h of each attention head.
2. The dimensions of Q , K , and V .
3. The dimensions of the attention matrix $QK^\top$ for one attention head.
4. The total number of parameters in:
 - W_Q , W_K , and W_V ,
 - W_O ,
 - the two MLP layers.
5. The MACs required for the Q , K , and V projections.
6. The MACs required to compute $QK^\top$ across all attention heads.
7. The MACs required to compute AV across all attention heads.
8. The MACs required for the attention output projection.
9. The MACs required for the MLP.
10. The total MACs and FLOPs for the complete Transformer block.

Part (b): KV Cache

Consider autoregressive inference for a single Transformer layer with

$$d = 768, \quad h = 12, \quad t = 512.$$

Assume FP16 tensors and batch size $B = 1$.

Deliverables. Compute and report:

1. The MACs required to compute K and V without a KV cache.
2. The MACs required to compute the new K and V vectors with a KV cache.
3. The reduction in K/V projection computation obtained using the cache.
4. The memory required to store the KV cache.

3 Task 3: Model Efficiency Metrics

Part (a): Matrix Multiplication

Consider

$$X \in \mathbb{R}^{32 \times 1024}, \quad W \in \mathbb{R}^{1024 \times 4096},$$

using FP16 precision.

Deliverables. Compute and report:

1. The dimensions of $Y = XW$.
2. The total number of parameters in W .
3. The memory required to store W .
4. The total number of MACs.
5. The total number of FLOPs.
6. The total number of bytes moved.
7. The arithmetic intensity in FLOPs/byte.

Part (b): Effect of Numerical Precision

Deliverables. Repeat the memory and arithmetic-intensity calculations for:

- FP32: 4 bytes per element,
- FP16: 2 bytes per element,
- INT8: 1 byte per element.

Assume the dimensions and number of arithmetic operations remain unchanged.

Complete and report the following table:

Precision	MACs	FLOPs	Bytes Moved	FLOPs/Byte
FP32				
FP16				
INT8				

Briefly explain why reducing numerical precision can reduce memory requirements while leaving the number of MACs unchanged.

4 Task 4: Profiling Techniques

Unless explicitly varied as part of an experiment, use batch size

$$B = 1.$$

Generate all inputs synthetically using PyTorch.

Part (a): CNN Profiling

Generate

$$X \in \mathbb{R}^{1 \times 64 \times 56 \times 56}.$$

For example,

```
x = torch.randn(1, 64, 56, 56, device=device)
```

Construct the following four models directly using PyTorch:

1. Standard 3×3 convolution:

$$64 \rightarrow 128.$$

2. Depthwise 3×3 convolution:

$$64 \rightarrow 64,$$

with

$$\text{groups} = 64.$$

3. Pointwise 1×1 convolution:

$$64 \rightarrow 128.$$

4. Depthwise separable convolution consisting of:

- depthwise 3×3 convolution with `groups=64`,
- pointwise 1×1 convolution from 64 to 128 channels.

Deliverables. For each model, measure and report:

- Parameters,
- MACs,
- FLOPs,
- Inference latency,
- Peak GPU memory usage.

Run several warm-up iterations before collecting measurements and average the results over multiple forward passes.

Part (b): Transformer Profiling

Construct a single Transformer block using

$$d = 768, \quad h = 12, \quad d_{\text{ff}} = 4d.$$

The block should contain:

- query, key, and value projections,
- multi-head self-attention,
- attention output projection,
- two-layer MLP.

You may ignore LayerNorm and dropout.

For each sequence length N , generate a synthetic FP16 tensor

$$X \in \mathbb{R}^{1 \times N \times 768}.$$

For example,

```
x = torch.randn(
    1, N, 768,
    device=device,
    dtype=torch.float16
)
```

Profile

$$N \in \{128, 256, 512, 1024\}.$$

Deliverables. For each sequence length, report:

- Parameters,
- MACs,
- FLOPs,
- Inference latency,
- Peak GPU memory usage.

Part (c): KV-Cache Profiling

Extend the same single Transformer block to perform autoregressive decoding.

Compare:

1. decoding without a KV cache,
2. decoding with a KV cache.

For the cached implementation, store the key and value tensors corresponding to previous tokens and reuse them during later decoding steps.

Profile

$$t \in \{128, 256, 512, 1024\}.$$

Deliverables. For both implementations, measure and report:

- Average latency per generated token,
- Total decoding latency,
- Peak GPU memory usage,
- KV-cache memory usage,
- Power consumption, where available,
- Energy per generated token, where available.

Part (d): Roofline Sweep

Construct a sequence of workloads whose arithmetic intensity increases across several orders of magnitude, and observe the transition from a memory-bound regime toward a compute-bound regime. Use the same device and numerical precision throughout this part.

Begin by characterizing the device. Measure:

- peak compute throughput P_{peak} , using a large square matrix multiplication at the precision under test,
- memory bandwidth β , using a streaming operation such as `torch.add(a, b, out=c)` on arrays totalling at least 256 MB, counting two reads and one write per element.

From these, compute the transition intensity

$$I^* = \frac{P_{\text{peak}}}{\beta}.$$

These values are measured rather than taken from device specifications. In general, the peak throughput reported in the device datasheet will upper-bound the measured value. Ensure that any reported peak throughput used for comparison corresponds to the same numerical precision as the experiment. Timing errors, rather than modelling errors, are a common reason a Roofline sweep shows no structure. Allocate all tensors and modules before the timed region. On CUDA, synchronize before and after the timed region, since kernel launches are asynchronous. Report the median of at least 20 timed iterations following at least 10 warm-up iterations.

```
for _ in range(10): # warm-up
    fn()

torch.cuda.synchronize()
times = []

for _ in range(20):
    torch.cuda.synchronize()
    t0 = time.perf_counter()
    fn()
    torch.cuda.synchronize()
    times.append(time.perf_counter() - t0)

latency = sorted(times)[len(times) // 2] # median
```

For the main sweep, perform square matrix multiplications

$$C = AB, \quad A, B, C \in \mathbb{R}^{N \times N},$$

and progressively increase N over a sufficiently wide range. For each N , compute

$$\text{FLOPs} = 2N^3, \quad \text{Bytes Moved} = b(3N^2),$$

where b is the number of bytes per element, and measure the achieved throughput.

Choose matrix sizes that span several orders of magnitude in arithmetic intensity where possible. Continue increasing the matrix size until throughput begins to saturate or available device memory becomes a limiting factor.

Compare the highest throughput obtained in the sweep with both the measured P_{peak} and the peak throughput reported in the device datasheet at the same numerical precision. If the sweep does not reach the measured or theoretical compute ceiling, report the fraction of peak throughput achieved and explain the likely reason for the shortfall.

Next, sweep the batch size of the standard convolution layer analysed in Task 1,

$$C_{\text{in}} = 64, \quad C_{\text{out}} = 128, \quad K = 3, \quad S = 1, \quad P = 1, \quad H = W = 56,$$

for

$$B \in \{1, 2, 4, 8, 16, 32, 64, 128\},$$

using the same precision as above. Task 1 evaluates this layer analytically at $B = 1$ only. Report $I(B)$ and the achieved throughput, and then:

1. state the range over which I varies across the sweep,
2. derive $\lim_{B \rightarrow \infty} I(B)$ analytically and compare it with your measurement,
3. compute the fraction of total traffic represented by the weight tensor at $B = 1$, and use it to explain the largest increase in I that batching this layer can produce,
4. explain why increasing batch size may substantially increase FLOPs without producing a corresponding increase in arithmetic intensity.

This sweep may produce a near-vertical cluster of points rather than a clear Roofline transition. Marks are awarded for correctly explaining why the transition is or is not observed.

Next, perform a wider sweep of the same convolution workload by varying the channel dimensions C_{in} , C_{out} , and kernel size K at fixed spatial resolution. Recompute FLOPs, bytes moved, and arithmetic intensity for every configuration rather than assuming that enlarging a dimension increases arithmetic intensity.

Choose configurations spanning as wide a range of arithmetic intensities as practical and determine whether the workload transitions toward compute-bound execution. If the transition is not observed, explain why based on the arithmetic-intensity range, measured throughput, and device compute limit.

Finally, perform a Transformer sequence-length sweep for both **prefill** and **decode**. Keep the model configuration fixed and vary the sequence length N over a suitable range. For each sequence length, recompute FLOPs, bytes moved, arithmetic intensity I , and achieved throughput for both phases.

Plot achieved throughput against arithmetic intensity for prefill and decode on the same Roofline figure. Compare how the two phases scale with increasing sequence length, including the growth of the KV cache, and explain whether either phase moves from a memory-bound toward a compute-bound regime. State the range of I obtained for each phase and whether it crossed I^* . **Include the corresponding Roofline plots for the matrix multiplication sweep, convolution sweeps, and Transformer prefill/decode sweep.**

Deliverables. Report, for every configuration in all sweeps: MACs, FLOPs, bytes moved, arithmetic intensity, median latency, achieved throughput, and include all relevant plots in your notebook.

Briefly discuss:

- which configurations are memory bound and which are compute bound,
- the arithmetic intensity at which the transition begins, and how it compares with I^* ,
- the fraction of measured and datasheet peak throughput reached by the highest-intensity configuration, and the reason for any shortfall,
- why the measured transition may be a gradual bend rather than the sharp corner of the idealized model,
- which parameter changes were most effective at increasing arithmetic intensity, and why some changes increased FLOPs substantially without a corresponding increase in I .

Recommended Profiling Resources

- PyTorch Profiler: [Official PyTorch Profiler Documentation](#)

- PyTorch Profiler Tutorial: Official PyTorch Profiler Tutorial
- PyTorch GPU Power Measurement: `torch.cuda.power_draw()` Documentation
- DeepSpeed Flops Profiler: DeepSpeed Flops Profiler Documentation

References and Useful Resources

- **Understanding Deep Learning – Simon J.D. Prince:**
Understanding Deep Learning – Book and Notebooks
- **Understanding Deep Learning GitHub Notebooks:**
Official UDL GitHub Repository
- **PyTorch Convolution Documentation:**
PyTorch Conv2d Documentation
- **PyTorch Transformer Documentation:**
PyTorch Transformer Documentation
- **PyTorch Multi-Head Attention:**
PyTorch MultiheadAttention Documentation
- **KV Cache – Hugging Face Transformers:**
Hugging Face KV Cache Explanation
- **KV Cache Strategies – Hugging Face Transformers:**
Hugging Face KV Cache Documentation
- **Roofline: An Insightful Visual Performance Model:**
Williams, Waterman, and Patterson, CACM 2009
- **NVIDIA Deep Learning Performance Guide:**
GPU Performance Background – Arithmetic Intensity and Tensor Cores
- **PyTorch Profiler:**
PyTorch Profiler Documentation
- **DeepSpeed Flops Profiler:**
DeepSpeed Flops Profiler Documentation